

Математическое моделирование персонально-ориентированного обучения и траектории обучения

Введение

Современную систему образования нельзя представить без инфраструктуры, многие составляющие которой имеют префикс «электронный» или «цифровой» (электронная дидактика, цифровая педагогика, электронная лекция, ...). Компьютерная поддержка обучения продолжает развиваться вместе с развитием информационных технологий. О поддержке конструктивизма и электронной дидактики и о подходе к дидактической инженерии изложено в [1, 2]. Розитса Донева и её коллеги [3] сравнивают традиционное и электронное обучение по основным элементам обучения. Там описываются преимущества и недостатки нескольких платформ обучения (Moodle, LAMS, PeU), представляются основные функциональные возможности системы BEST, которая включает в себя все положительные стороны предыдущих.

Современный подход к обучению представляет обучение как конструирование знаний и включает в себя понятие формального представления знаний, которое давно уже существует как большая отрасль искусственного интеллекта [4, 5]. Одним из быстро развивающихся направлений представления (моделирования) знаний в компьютере являются семантические сети [6, 7]. Как и другие информационные технологии, семантические сети тоже применяются в образовании. В работе Кристансона и его коллег [8] излагается решение задачи обнаружения пробелов (пропусков) знаний в учебниках на примере 10 учебников по линейной алгебре на основе семантических сетей.

Доунс [9] объясняет объекты обучения (Learning Object) с метаданными для поддержки повторной используемости и доступности для программных агентов обучения. Он также поддерживает подход, в котором создание электронного курса, подобно разработке программного обеспечения, должно конструироваться с соблюдением:

- 1) концепции RAD, где продукт разрабатывается с использованием повторно используемых компонентов. Онлайн-курс рассматривается как система, сконструированная из повторно используемых объектов обучения;
- 2) концепции объектно-ориентированного программирования, где объекты обучения будут создаваться на основе некоторого класса в иерархии классов и могут состоять в некотором отношении (ассоциации, агрегации, ...) с другими объектами — как объект теста, курса и т.д.;
- 3) концепции открытых стандартов (LOM, SCORM), чтобы распределённые среды обучения и объекты обучения, разработанные разными авторами на разных языках, платформах и находящиеся на разных компьютерах, смогли работать совместно [16].

Речь в основном идёт о создании курса; семантическая информация (метаданные) используется для интеграции объектов обучения в электронный курс, для описания иерархической структуры. Формализм для описания метаданных — как и в IMS, SCORM [13] — предлагается XML, XSL и CSS.

В статье Арнольда Кива и его коллег [14] рассказывается о семантической сети обучения, об алгоритме её построения и её визуализации. Приводится алгоритм

определения логической последовательности между основными концептами предмета. В частности, рассматриваются семантические сети для предметов «Экономическая кибернетика», «Алгоритмы и программирование» и «Вычислительная математика». Рассказывается о топологических характеристиках сети на основе таких свойств, как степень вершины, центральность, кластерность и т.д.

Современная глобальная сеть (Web3) включает в себя семантическую составляющую и называется семантическим вебом [15]. Многие образовательные системы разработаны на основе данного типа сети и называются семантической сетью обучения (Semantic Learning Web). Основные принципы функционирования таких систем описаны в работах Стутта, Мотты и Деведжика [10, 11].

Чтобы интегрировать учебный курс и учебную платформу, разработанные не знакомыми друг с другом педагогом и программистом, чтобы реализовать интерактивность курса и получить необходимую обратную связь, были разработаны стандарты. В стандарте SCORM детально описываются соответствующие механизмы [13]; они совершенствовались по мере развития технологий [18].

Данная статья является попыткой математического моделирования процесса персонально-ориентированного обучения [19], которое, в свою очередь, является составляющим звеном конструктивизма [20].

На основе нижеописываемой модели можно специфицировать программную систему для машинной поддержки интеллектуального анализа процесса (траектории) обучения — выявлять правила (ассоциации), сходства, различия и аномальные явления в используемой дидактике и в траекториях обучения обучаемых [21, 22].

Когда дидактически переработанный учебный материал обладает свойством настраиваться в зависимости от потребностей обучаемого для повышения качества (приближения результата освоения к дидактической цели) и эффективности освоения (по времени и по другим всевозможным ресурсам), тогда мы говорим о персонально-ориентированном обучении [19].

В данной статье рассматривается процесс обучения как объект исследования. При этом дидактическая цель представляется как знание (Objective — O). Текущий уровень освоения обучаемого (запоминание, понимание, применение, анализ, синтез, сравнительная оценка, мастерство, владение [5]) тоже рассматривается как знание (Comprehension — C).

Знание (внутреннее представление в памяти компьютера) имеет вид некоторого помеченного графа, где вершины соответствуют определённым фактам или обобщённым концептам, дуги означают отношения или ассоциации между вершинами (онтология, семантическая сеть, фрейм, ...) [6]. Далее по тексту, когда речь идёт о знании, если нет двусмысленности, то имеем в виду некоторый граф, соответствующий знанию.

Качество обучения представляется как некоторое значение, семантически означающее близость, схожесть или расстояние (Distance — D) между уровнем освоения обучаемого C и целью обучения O. Для заданных двух графов C и O расстояние можно определить [21] на основе:

- максимального расстояния общих подграфов;

- схожести структур;
- операций редактирования;
- ядерных функций.

Так как речь идёт о конструировании знания, наиболее подходящим является метод на основе операций редактирования, алгоритмы которых подробно описывает Аггарвал в гл. 17 своего учебника [21].

1. Репозиторий объектов обучения

Объект обучения — это объяснение некоторого контента — SCO [7] учебного материала вместе с необходимыми метаданными для взаимодействия с платформой обучения. SCO может быть элементарным активом (asset: текст, процедура, рисунок, аудио, видео, ...) или некоторой комбинацией элементарных активов и SCO. Доунс [9] называет такие объекты объектами обучения (Learning Objects).

Репозиторий объектов обучения (R) — это хранилище, содержащее множество объектов обучения, структурированных по метаданным. Каждый объект обучения в репозитории описывается тройкой вида $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$, где *Pre* — предварительное знание, необходимое для понимания объяснения, *Exp* — само объяснение (учебный контент), *Res* — знание, которым овладевает обучаемый после изучения данного объекта. Репозиторий служит основой для автоматического конструирования дидактики: алгоритм поиска подходящего объекта обучения опирается на сопоставление требуемого предварительного знания и результата объекта с заданной дидактической целью.

2. Дидактика

Моделируем понятия дидактики для обучения некоторому знанию.

Определение (Дидактика): Дидактикой обучения (далее — просто дидактика) назовём тройку $D = \langle \text{Pre}, \text{Res}, G \rangle$, где *Pre* — некоторое предварительное знание, необходимое для обучения данному знанию; *Res* — дидактическая цель обучения — знание, которое получает обучаемый в результате обучения на основе данной дидактики; $G = \langle N, E \rangle$ — дидактический граф — ориентированный граф дидактически переработанного контента. *N* — множество вершин графа; *E* — множество дуг графа.

Вершинам дидактического графа *G* соответствуют события в учебном процессе, то есть к вершинам приписываются тройки вида $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$, где:

Exp (explanation) — объект обучения;

Pre (Precondition) — предусловие — некоторое предварительное знание, требуемое для понимания *Exp*, то есть объяснение базируется на данном знании;

Res (Result) — постусловие, результат — некоторое знание, которым будет (должен) овладеть обучаемый в результате объяснения *Exp*.

Семантика вершины означает, что «если обучаемый знает *Pre*, то он (она) после объяснения *Exp* будет знать *Res*».

Дугам дидактического графа *G* соответствуют отношения «следования» среди вершин графа. Если существует дуга по направлению из вершины n_i к вершине n_j ,

это означает, что объяснение $n_j.\text{Exp}$ следует после (базируется на) объяснения $n_i.\text{Exp}$.

Для вершины n всегда выполняется $n.\text{Pre} \subseteq n.\text{Res}$, и $\Delta n = n.\text{Res} - n.\text{Pre}$ является тем знанием, которое получает обучаемый после объяснения $n.\text{Exp}$, и называется дидактическим эффектом вершины n .

Предполагаем, что для $\forall$ вершины n дидактики выполняются следующие условия:

- 4) $\forall j \in \text{predecessor}(n), \neg(n.\text{Pre} \subseteq (\cup_{i \in \text{predecessor}(n)} i.\text{Res} - \Delta j))$ — условие необходимости любого предшественника;
- 5) $n.\text{Pre} \subseteq \cup_{i \in \text{predecessor}(n)} i.\text{Res}$ — условие достаточности предшественников;
- 6) $\cup_{i \in \text{predecessor}(n)} i.\text{Res} = \min\{\exists x \in \mathcal{R}(N) : n.\text{Pre} \subseteq \cup_{i \in x} i.\text{Res}\}$ — условие минимальности предшественников.

Здесь: $\text{predecessor}(n)$ — множество предшественников вершины n ; $\mathcal{R}(N)$ — множество всех подмножеств множества N .

Условие 1 говорит, что вершина j необходима для понимания (освоения) $n.\text{Exp}$, то есть без усвоения объяснения, находящегося в предшественнике j , нельзя освоить преемника n . Условие 2 говорит, что для усвоения преемника достаточно освоить всех его предшественников, то есть предшественники n достаточны для освоения объяснения n . Условие 3 говорит, что множество предшественников вершины составляет минимальный объём знаний, требуемый для освоения объяснения преемника, то есть любые другие множества вершин, освоение которых достаточно для освоения n , содержат больше знаний, чем знания, содержащиеся во всех предшественниках n .

С точки зрения конструирования дидактики, вершина подвешивается как преемник к минимальному количеству вершин, от которых она логически зависит; одновременно предшественники вместе составляют достаточное знание для понимания объяснения вершины n .

Начальной вершиной дидактики является та, у которой нет предшественников. Может быть так, что в дидактике есть несколько начальных вершин — $n_1, n_2, \dots, n_k$. Тогда можно свести их к одной начальной вершине — n_s , соединив $n_1, n_2, \dots, n_k$ как преемники n_s , и установив $n_s.\text{Res} = \cup_{i \in \{n_1, \dots, n_k\}} j.\text{Pre}$ с пустым дидактическим эффектом ($\Delta n_s = \emptyset$), то есть $n_s.\text{Pre} = n_s.\text{Res}$. Далее, не нарушая общности, считаем, что дидактический граф G имеет единственную начальную вершину n_s (обозначаем как $G.n_s$) с предусловием дидактики $G.\text{Pre}$. Назовём его предусловием дидактики $D.\text{Pre}$.

Конечной вершиной дидактики является та, у которой нет преемников. Точно так же в дидактике может быть несколько конечных вершин $n_1, \dots, n_k$. Их также можно свести к одной — n_f , соединив $n_1, \dots, n_k$ как предшественники n_f , и установив $n_f.\text{Pre} = \cup_{i \in \{n_1, \dots, n_k\}} j.\text{Res}$ с пустым дидактическим эффектом ($\Delta n_f = \emptyset$), то есть $n_f.\text{Res} = n_f.\text{Pre}$. Далее, не нарушая общности, считаем, что дидактика имеет единственную конечную вершину n_f (обозначаем как $G.n_f$) с результатом дидактики Res . Обозначаем результат дидактики как $D.\text{Res}$.

На рис. 1 представлен дидактический граф, где четыре начальные вершины (a, b, c, d) приведены к одной (n_s), и две конечные вершины (j, k) приведены к одной (n_f).

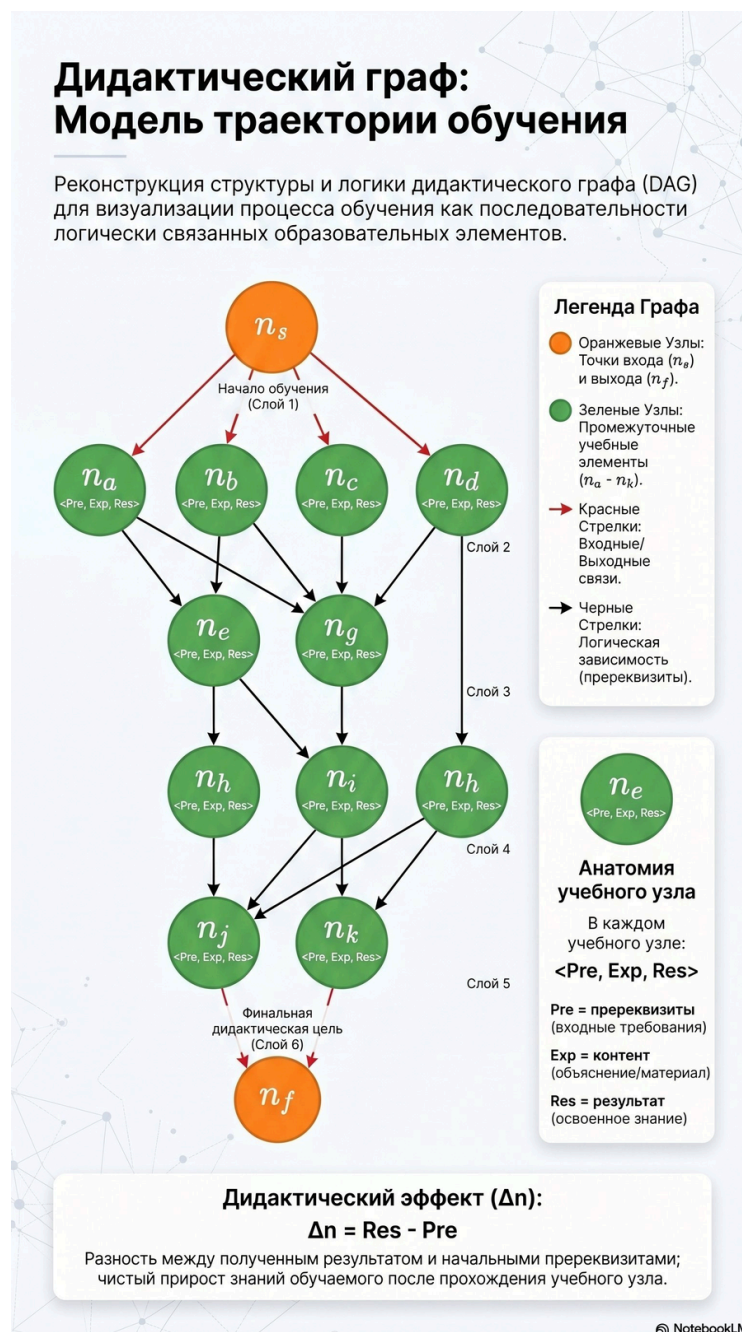


Рис. 1. Дидактический граф.

3. Конструирование дидактики

Допустим, что в репозитории R существует достаточное количество учебных объектов для заданной цели обучения. Рассмотрим задачу подвешивания новой вершины.

3.1. Определение предшественников вершины

Определение предшественников вершины состоит из двух алгоритмов: FindPredecessors и CreatePredecessors. Алгоритм CreatePredecessors формирует список всех потенциальных предшественников PotentialList, затем вызывает

FindPredecessors для определения оптимального списка предшественников OptimalList.

FindPredecessors определяет такой список предшественников, в котором не пересекаются дидактические эффекты, то есть не будет двух вершин, которые повторяют некоторое знание. Для этого используется переменная Δ_{hor} , которая аккумулирует дидактические эффекты обнаруженных предшественников. Кроме того, суммарное знание в OptimalList будет минимально покрывающим, то есть без лишних знаний для освоения объяснения вершины n . После этого CreatePredecessors создаёт вершины-предшественники для каждого члена из списка OptimalList и (если есть) дополнительную вершину для оставшегося недостающего знания $Pre - MinRes$, необходимого для освоения объяснения в n .

Глобальные переменные, используемые в алгоритмах:

- Список всех потенциальных предшественников — PotentialList;
- Требуемое общее знание для предшественников — Pre;
- Минимальное объединение результатов предшественников — MinRes;
- Оптимальный список предшественников — OptimalList.

Алг. FindPredecessors (n , Res, Δ_{hor} , MinRes, CurPred)

Вход: Текущая вершина в списке PotentialList — n , текущее объединение результатов предшественников — Res, текущее объединение эффектов предшественников — Δ_{hor} , текущий список предшественников — CurPred.

Выход: Оптимальный список предшественников — OptimalList.

Метод:

$m = \text{Next}(\text{PotentialList}, n);$

Если $\Delta_{hor} \cap \Delta_n = \emptyset$ то // Условие горизонтального непересечения

$y = \text{Res} \cup n.\text{Res}; \text{Incl}(\text{CurPred}, n);$

Если $y \supseteq \text{Pre}$ то // Количество предшественников достаточно

Если $y \subset \text{MinRes}$ то // Корректировка оптимального списка

$\text{MinRes} = y; \text{OptimalList} = \text{CurPred};$

Всё

Иначе

Если $m \neq \text{null}$ то

$\text{FindPredecessors}(m, y, \Delta_{hor} \cup \Delta_n, \text{CurPred});$ // С учётом вер. n

Иначе

Если $S = \emptyset$ то $\text{MinRes} = y; \text{OptimalList} = \text{CurPred};$ Всё

Всё

Всё

$\text{Excl}(\text{CurPred}, n);$

Всё

Если $m \neq \text{null}$ то

FindPredecessors (m , Res, Δ_{hor} , CurPred); // Без учёта вер. n

Всё

Важным моментом является то, чтобы предусловие ($n.\text{Pre}$) не пересекалось с объяснением, то есть чтобы объяснение прямо или косвенно не основывалось само на себе. Для этого в CreatePredecessors используется переменная Δ_{ver} , которая аккумулирует дидактические эффекты по пути предшествования.

Алг. CreatePredecessors (n , D , R , Δ_{ver})

Вход: Вершина, для которой определяются предшественники — n ; текущий (построенный до сих пор) дидактический граф — D ; репозиторий элементарных учебных контентов — R ; объединение дидактических эффектов по пути предшествования — Δ_{ver} .

Выход: Множество S — всех предшественников вершины n и недостающее знание m .

Метод:

$\text{Pre} = n.\text{Pre}$; $\text{OptimalList} = \emptyset$; $\text{PotentialList} = \emptyset$;

Для $\forall d \in B$ (сначала по D , потом по $R-D$) выполнить

Если $(\Delta d \cap \Delta_{\text{ver}} = \emptyset) \ \& \ (\Delta d \cap \text{Pre} \neq \emptyset)$ то $d \rightarrow \text{PotentialList}$

Всё

FindPredecessors($\text{PotentialList.first}$, $\emptyset$, $\emptyset$, PotentialList , $\emptyset$);

Для $\forall k \in \text{OptimalList}$ выполнить

GenPred(n , k);

CreatePredecessors(k , D , R , $\Delta_{\text{ver}} \cup \Delta k$)

Всё

Если $\text{MinRes} \subset \text{Pre}$ то

$n_s.\text{Res} = n_s.\text{Res} \cup \text{Pre} - \text{MinRes}$;

GenPred(n , n_s);

Всё

Используемые дополнительные функции:

- Incl(List, n) — включает элемент n в список List;
- Excl(List, n) — исключает элемент n из списка List;
- Next(List, n) — выдаёт следующий элемент после элемента n из списка List;
- GenPred(n , k) — создаёт дугу (k , n);
- CreateNode(Pre , Exp , Res) — создаёт вершину с тройкой $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$;

- $\text{Predecessors}(n)$ — список всех предшественников вершины n .

3.2. Генерация дидактики

Генерация дидактики основывается на объекте обучения. Алгоритм *DidacticGen* создаёт начальную и конечную вершины с пустым дидактическим эффектом и формирует всех предшественников конечной вершины, вызывая *CreatePredecessors*.

Алг. *DidacticGen* (O, R)

Вход: Репозиторий элементарных учебных контентов — троек $\langle \text{Pre}, \text{Exp}, \text{Res} \rangle$ — R и дидактическая цель (Objective) — O .

Выход: Дидактика D , где $D.\text{Res} \supseteq O$.

Метод:

$n_f = \text{CreateNode}(O, \emptyset, O); D = n_f;$

$n_s = \text{CreateNode}(\emptyset, \emptyset, \emptyset);$

$\text{CreatePredecessors}(n_f, D, R, \emptyset);$

Утверждение 1. Дидактика, построенная вышеуказанным алгоритмом, является ациклической.

Доказательство. Допустим, существует некоторый цикл. Тогда можно представить некоторую последовательность вершин $e_{i1}, e_{i2}, \dots, e_{ik}$, составляющую путь, где $e_{i1} = e_{ik}$. Тогда, так как в алгоритме *CreatePredecessors* переменная Δver аккумулирует дидактические эффекты по пути и не допускает, чтобы дидактический эффект очередной вершины по пути пересекался с Δver , получается, что дидактический эффект вершины добавляется дважды. Это противоречие доказывает данное утверждение. ■

Поддидактика: Дидактика D_1 является поддидактикой D_2 , если $D_1.O \subseteq D_2.O$ и $D_1 \subseteq D_2$ в смысле графа, то есть $N_{D_1} \subseteq N_{D_2} \ \& \ E_{D_1} \subseteq E_{D_2}$.

Эквивалентная дидактика: Дидактики D_1 и D_2 являются эквивалентными, если $O_{D_1} = O_{D_2} \ \& \ D_1.\text{Pre} = D_2.\text{Pre}$.

Детализированная вершина: Для вершины n дидактики D_1 , дидактика D_2 является детализацией, если n и D_2 являются эквивалентными.

Детализированная дидактика: Для заданных дидактик D_1 и D_2 , говорим, что D_2 детализирует D_1 , если существует хотя бы одна вершина в D_1 , которую D_2 детализирует, а в остальном они совпадают.

4. Обучение

На данный момент генерация дидактики является нелёгкой (по времени и по объёму) работой для обычных серверов. Поэтому однажды сгенерированная дидактика обычно не удаляется после использования, а сохраняется в репозитории для дальнейшего повторного использования. Таким образом, при поиске элемента обучения в репозитории существует некоторая вероятность найти уже готовую дидактику.

Время от времени необходимо перестраивать хранимую дидактику за счёт новодобавленных элементов обучения и дидактик. Однако данная проблема не входит в рамки настоящей статьи.

Когда при поиске в репозитории обнаруживается дидактика, цель которой покрывает заданную, тогда необязательно строить дидактику с нуля — можно использовать найденную. Далее моделируем процесс обучения обучаемого на основе заранее сконструированной дидактики.

Необязательно, чтобы обучаемый изучил все учебные материалы, приписанные вершинам дидактики, если цель обучения является подмножеством цели сконструированной дидактики. Он (она) может начать не с начала, так как уже знаком(а) с некоторой частью материала, или может не изучать до конца, то есть имеет цель обучения, являющуюся подцелью цели данной дидактики. Таким образом, нужно будет «пройти» не всю дидактику, а некоторую поддидактику. Обучаемый последовательно изучает объяснения всех вершин поддидактики в некотором порядке. На основе условия 2 дидактики обучаемый освоит учебный материал, сначала изучая всех предшественников вершины, а затем саму вершину-преемника. Порядок изучения самих предшественников при этом не важен.

Определение (Обучение): Обучением называем такую линеаризацию (нумерацию) вершин поддидактики, при которой не нарушается вышеуказанное условие.

Допустим, обучаемый (L) имеет цель — достижение некоторого знания ($L.O$) — с предварительной подготовкой (осведомлённостью) $L.C$. При этом считаем, что $L.C \subset L.O$, иначе в обучении нет смысла. Для того чтобы воспользоваться некоторой сконструированной дидактикой D , необходимо чтобы $(D.Pre \subseteq L.C) \ \& \ (D.O \supseteq L.O)$, то есть обучаемый смог освоить учебный материал дидактики и после освоения достиг своей цели. Назовём такую дидактику подходящей для обучаемого L .

4.1. Определение минимальной цели, удовлетворяющей обучаемого

Алгоритм определения минимальной цели, удовлетворяющей обучаемого, на основе параметров O — объекта обучения и D — дидактики — определяет такую вершину n дидактики D , что $n = \text{Min}(k.Res : \forall k \in D.N) \ \& \ O \subset n.Pre$. Алгоритм просматривает все вершины графа D , начиная с $D.n_f$, снизу вверх по всем предшественникам, результат которых покрывает O .

Алг. MinObjective (O, D, m)

Вход: Цель ($L.O$) и дидактика D , такие что $L.O \subset D.Res$.

Выход: Вершина $\text{minObjective} \in D.N$, такая что $L.O \subseteq \text{minObjective}.Res$, и для $\forall n \in D.N$, где $\text{minObjective} \neq n$: $\text{minObjective}.Res \subseteq n.Res$ или $n.Res \subset L.O$. То есть minObjective является минимально покрывающей $L.O$ вершиной. Минимальная поддидактика $D_{\min}$, которая обеспечивает $L.O$.

Метод: Пусть Q — некоторая очередь.

$m = D.n_f; \text{Incl}(Q, D.n_f);$

Пока $Q \neq \emptyset$ выполнить

$n = \text{Excl}(Q, n);$

Если $n.\text{Res} \subset \text{minObjective}.\text{Res}$ то $\text{minObjective} = n; \text{Всё}$

Для $\forall k \in \text{Predecessors}(n)$ выполнить

Если $k \notin Q$ то $\text{Incl}(Q, k); \text{Всё}$

Всё

Всё

Заметим, что здесь не следует прекращать поиск по предшественникам вершины, если Res вершины не покрывает $L.O$, потому что в Res включаются знания $\text{Pre} + \Delta$, а Pre является подмножеством Res своих предшественников; таким образом, среди предшественников может существовать такой, что его Res охватит $L.O$.

Если для заданных L и D , $\text{MinObj}(L.O, D) = D.n_s$, то это означает, что данная дидактика слишком сложна для данного обучаемого и для её изучения требуется предварительная подготовка.

5. Дерево обучения

Обучение обучаемого по дидактике доходит до вершины с минимальной целью, удовлетворяющей обучаемого (покрывающей цель обучаемого, $\text{Res} \subseteq L.O$), а начинается с вершин, которые приводят к минимальной цели и являются достаточно простыми для освоения обучаемого (покрываются осведомлённостью обучаемого, $\text{Pre} \subseteq L.C$). После определения минимальной цели обучения определяется минимальная поддидактика, необходимая для обучения. Именно эту минимальную поддидактику и должен «пройти» обучаемый для достижения цели $L.O$.

Алгоритм определения дерева обучения LearnTree определяет некоторое дерево, которое служит для достижения обучаемым цели обучения.

Алг. $\text{LearnTree}(L.C, D, \text{minObjective})$

Вход: Предварительная подготовка $L.C$, дидактика D с вершиной minObjective .

Выход: Подграф, образованный на основе отмеченных вершин дидактики.

Метод: Пусть Q — некоторая очередь.

$\text{Incl}(Q, \text{minObjective});$

Пока $Q \neq \emptyset$ выполнить

$n = \text{Excl}(Q, n);$ Отметить n ;

Для $\forall k \in \text{Predecessors}(n)$ выполнить

Если $(L.C \subset k.\text{Pre})$ то

Если $k \notin Q$ то $\text{Incl}(Q, k); \text{Всё}$

Иначе Отметить n ;

Всё

Всё

PostOrder(minObjective);

Здесь PostOrder(n) — восходящая (левосторонняя или правосторонняя) нумерация [24] ациклического графа, начиная с вершины n. PostOrder нумерует вершины подобно дереву, с одним отличием: при встрече с уже пронумерованной вершиной возвращается назад. Если удалить дуги, связывающие такие вершины, то получим дерево. Назовём такое дерево LearnTree(L.C, D, minObjective) — деревом обучения.

Например, если пронумеровать следующий ациклический граф посредством PostOrder, то получим дерево.

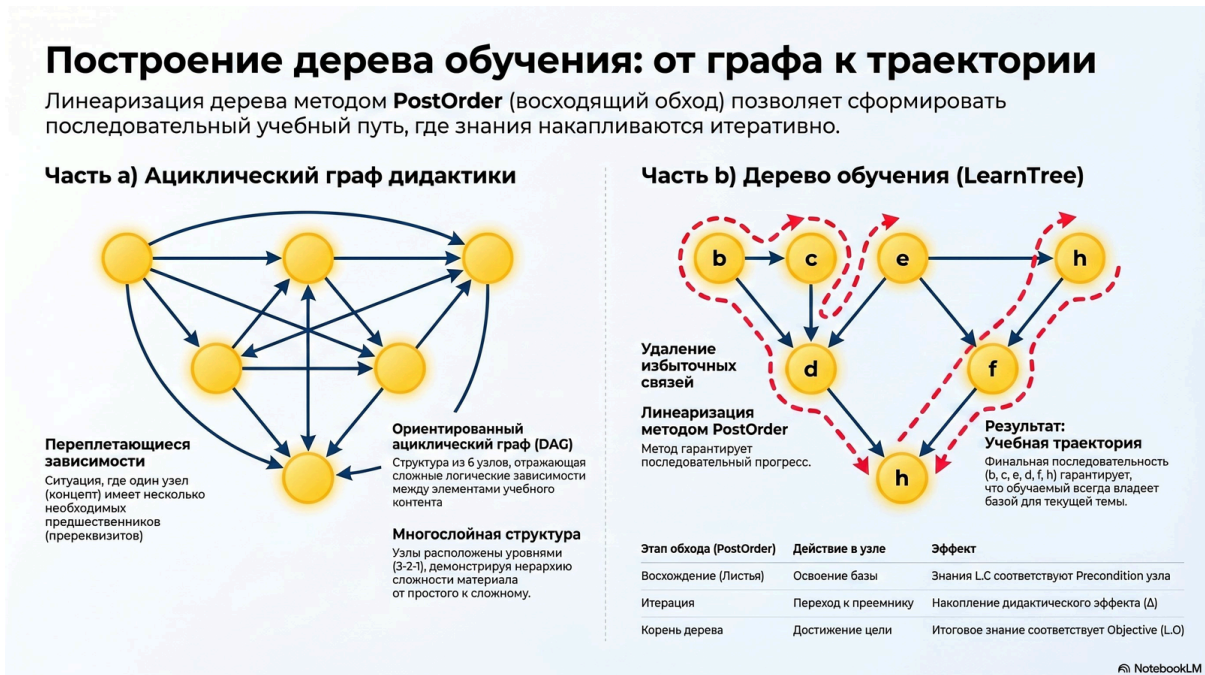


Рис. 2. а) ациклический граф, б) дерево, полученное нумерацией PostOrder.

Утверждение 2. Линеаризация дерева обучения посредством PostOrder является обучением.

Доказательство. Так как PostOrder — восходящий, то номер преемника всегда будет больше, чем любого предшественника. При разветвлении вершины ациклического графа сначала нумеруются её предшественники, а затем она сама — при первом обращении к ней из её преемника. При последующих обращениях по другим преемникам она игнорируется. Таким образом, любой преемник имеет больший номер, и это означает, что PostOrder является линеаризацией, при которой выполняется условие обучения. Это доказывает утверждение. ▮

Если обучаемый изучает материалы, приписанные вершинам дерева обучения, обходя все вершины дерева, предварительно пронумеровав их посредством PostOrder, то для любой вершины n дерева, изучение которой начинает обучаемый, выполняется $L.C \geq n.Pre$. Это означает, что обучаемый не попадает в ситуацию, где текущее объяснение основывается на материале, который он ещё не изучал.

Если посмотреть дерево обучения заданной дидактики — $LearnTree(D.Pre, D, MinObj(D.O, D))$, то, линеаризуя (последовательно сцепляя) учебные материалы,

приписанные вершинам дерева обучения согласно PostOrder, получим дидактически переработанный **электронный учебник**.

Таким образом, обучаемый достигает своей цели, изучая материалы, приписанные вершинам дерева обучения, обходя все вершины дерева согласно PostOrder, начиная с его листьев. При этом знания обучаемого накапливаются итеративно, вершина за вершиной: каждый раз, проходя вершину n , обучаемый приобретает приращение Δn . Можно говорить, что данная модель соответствует итеративному (гибкому) обучению [26].

Литература

1. Clark R.C., Mayer R.E. e-Learning and the Science of Instruction. Proven Guidelines for Consumers and Designers of Multimedia Learning. 3rd ed. Pfeiffer, 2011. 527 p.
2. Tchoshanov M. Engineering of Learning: Conceptualizing e-Didactics. UNESCO Institute for Information Technologies in Education, Moscow, 2013. 194 p.
3. Doneva R., Denev D., Totkov G. On the Didactic Principles, Models and E-Learning. International Journal «Information Theories and Applications».
4. Brachman R.J., Levesque H.J. Knowledge Representation and Reasoning. Morgan Kaufmann, 2004. 381 p.
5. Bloom B.S., Engelhart M.D., Furst E.J., Hill W.H., Krathwohl D.R. Taxonomy of Educational Objectives: The Classification of Educational Goals. Vol. Handbook I: Cognitive Domain. New York: David McKay Company, 1956.
6. Sowa J.F. (ed.) Principles of Semantic Networks: Explorations in the Representation of Knowledge. Morgan Kaufmann Publishers, 1991. 576 p.
7. Lehmann F. Semantic Networks. Computers Math. Applic. Vol. 23, No. 2–5, pp. 1–50, 1992.
8. Christianson N.H., Blevins A.S., Bassett D.S. Architecture and evolution of semantic networks in mathematics texts. Proc. R. Soc. A 476: 20190741. <http://dx.doi.org/10.1098/rspa.2019.074>
9. Downes S. Learning Objects: Resources For Distance Education Worldwide. International Review of Research in Open and Distance Learning. Vol. 2, No. 1, 2001.
10. Stutt A., Motta E. Semantic Learning Webs. Journal of Interactive Media in Education, Special Issue on the Educational Semantic Web, No. 10 (2004).
11. Devedzic V. Education and the Semantic Web. International Journal of Artificial Intelligence in Education, 14, 39–66 (2004).
12. IEEE LTSC: IEEE Learning Technology Standards Committee Glossary. IEEE P1484.3 Glossary Working Group, draft standard (2001).
13. ADL Initiative. Sharable Content Object Reference Model (SCORM). Advanced Distributed Learning Initiative, 2004. Available online at <http://www.adlnet.org/>
14. Kiv A., Soloviev V., Tarasova E., Koycheva T., Kolesnykova K. Semantic knowledge networks in education. E3S Web of Conferences 166, 10022 (2020) ICSF 2020. <https://doi.org/10.1051/e3sconf/202016610022>

15. Berners-Lee T., Hendler J., Lassila O. The Semantic Web. Scientific American, May 2001.
16. Hodgins W. Out of the past and into the future: Standards for technology enhanced learning. In: Ehlers U., Pawlowski J. (eds.). Handbook on Quality and Standardisation in E-Learning, pp. 309–327. Springer Berlin, 2006.
17. Tane J., Schmitz C., Stumme G., Staab S., Studer R. The Courseware Watchdog: an Ontology-based tool for finding and organizing learning material. In: Fachtagung Mobiles Lernen und Forschen, Kassel, 2003.
18. Advanced Distributed Learning. SCORM 2004 4th Edition Overview. ADL Initiative, 2009.
19. Weimer M. Learner-Centered Teaching: Five Key Changes to Practice. 2nd ed. San Francisco: Jossey-Bass, 2013. 288 p.
20. Papert S., Harel I. Constructionism. Ablex Publishing Corporation, 1991.
21. Aggarwal C.C. Data Mining: The Textbook. Springer, 2015. 746 p.
22. Tan P., Steinbach M., Karpatne A., Kumar V. Introduction to Data Mining. 2nd ed. New York: Pearson Education, 2019. 1459 p.
23. Bloom B.S. et al. — см. [5].
24. Aho A.V., Hopcroft J.E., Ullman J.D. Data Structures and Algorithms. Addison-Wesley, 1983.
25. Piaget J. The Equilibration of Cognitive Structures: The Central Problem of Intellectual Development. University of Chicago Press, 1985.
26. Sharples M. et al. Agile Methods for Technology-Enhanced Learning. In: Proceedings of the European Conference on Technology Enhanced Learning, 2014.